

모듈화

□ Node.js 모듈이란?

○ 모듈의 개념

- Node.js에서 하나의 JS 파일은 하나의 모듈로 간주됨
- 기능별로 코드를 분리하고 **export**, **import**를 통해 재사용 가능

○ 모듈이 필요한 이유

- 유지보수가 쉬움
- 재사용성이 높아짐
- 협업 및 기능 분리에 적합함

○ 모듈의 종류

- 함수 모듈 : 계산기, 인사 메시지 등
- 라우터 모듈 : 요청 경로에 따른 응답 분기
- 유틸리티 모듈 : 날짜 처리, 포맷 변환 등 자주 사용하는 공통 기능

[실습]

인사 메시지를 반환하는 함수 모듈 작성하기

1. 모듈 파일 - greet.js

```
export function getGreeting(name) {  
  return `안녕하세요, ${name}님!`;  
}
```

2. 모듈을 불러들일 파일 - main.js

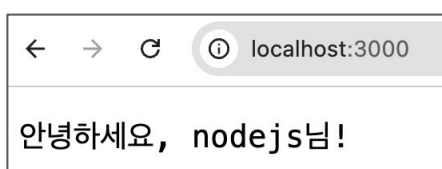
```
import http from 'node:http';  
import { getGreeting } from './greet.js'; // greet.js 모듈을 불러와서 사용
```

```
const server = http.createServer((req, res) => {  
  const name = 'nodejs';  
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf8' });  
  res.end(getGreeting(name));  
});
```

```
server.listen(3000, () => {  
  console.log('http://localhost:3000 에서 인사 메시지를 확인하세요');  
});
```

3. 동작 확인

node main.js



□ 모듈은 한 번만 로드 (모듈 캐싱)

○ 개념 정의

- Node.js는 **import** 또는 **require()**로 로드한 모듈을 메모리에 캐싱
- 한 번 불러온 모듈은 다시 읽지 않고 그대로 재사용

[실습]

모듈의 출력문이 한번만 동작하는 것을 확인

1. 모듈 파일 - log.js

```
console.log('log.js가 실행됨');
```

```
export const msg = 'Hello';
```

2. 모듈을 실행할 파일 - main.js (코드 추가)

```
import http from 'node:http';
```

```
import { getGreeting } from './greet.js'; // greet.js 모듈을 불러와서 사용
```

```
import { msg } from './log.js';
```

```
import { msg as again } from './log.js'; // 캐시되어 있어 다시 불러도 동작하지 않음
```

```
const server = http.createServer((req, res) => {  
  const name = 'nodejs';  
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf8' });  
  res.end(getGreeting(name));  
});
```

```
server.listen(3000, () => {  
  console.log('http://localhost:3000 에서 인사 메시지를 확인하세요');  
});
```

3. 동작 확인

node main.js

```
PROBLEMS  OUTPUT  TERMINAL  ...  
  
○ (base) ggoreb@GGoRebui-MacBookAir 03 % node main.js  
log.js가 실행됨  
http://localhost:3000 에서 인사 메시지를 확인하세요  
█
```

[실습]

유틸리티 모듈 구성 예시

1. 모듈 파일 - utils/date.js

// 현재 날짜를 형식에 맞춰 반환하는 유틸 함수

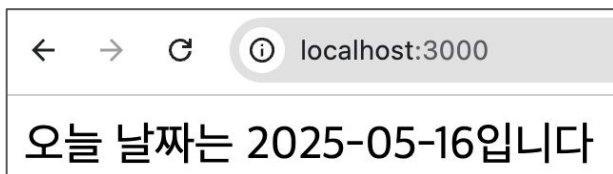
```
export function getToday() {  
  const now = new Date();  
  return now.toISOString().split('T')[0]; // YYYY-MM-DD 형식  
}
```

2. 모듈을 실행할 파일 - server.js

```
import http from 'node:http';  
import { getToday } from './utils/date.js';  
  
const server = http.createServer((req, res) => {  
  const today = getToday();  
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf8' });  
  res.end(`오늘 날짜는 ${today}입니다`);  
});  
  
server.listen(3000, () => {  
  console.log('http://localhost:3000 에서 실행중');  
});
```

3. 동작 확인

node server.js



[실습]

라우팅 모듈 구성 예시

1. 모듈 파일 - routes.js

// 요청 URL에 따라 분기해주는 함수

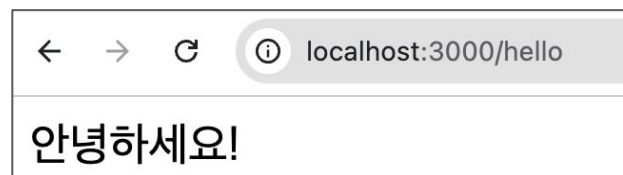
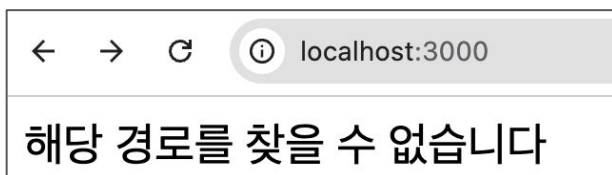
```
export function handleRoute(req, res) {  
  if (req.url === '/hello') {  
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf8' });  
    res.end('안녕하세요!');  
  } else {  
    res.writeHead(404, { 'Content-Type': 'text/html; charset=utf8' });  
    res.end('해당 경로를 찾을 수 없습니다');  
  }  
}
```

2. 모듈을 실행할 파일 - server-routes.js

```
import http from 'node:http';  
import { handleRoute } from './routes.js';  
  
const server = http.createServer((req, res) => {  
  handleRoute(req, res);  
});  
  
server.listen(3000);
```

3. 동작 확인

node server-routes.js



연습문제

1. 계산기 모듈 작성하기 ([math.js](#) / [calculator.js](#))

덧셈과 뺄셈 기능을 별도의 모듈로 분리하고, 이를 다른 파일에서 불러와 사용하는 프로그램을 작성합니다.

- 설명: 기능을 하나의 파일에 다 몰아넣지 않고 "계산 기능"처럼 역할이 분명한 코드는 별도의 모듈 파일로 분리해두면 나중에 재사용하거나 유지보수하기 훨씬 쉬워집니다.

- 요구사항

- `calculator.js` 모듈 작성
 - `add(a, b)`, `sub(a, b)` 함수 정의
 - `math.js`에서 `import`하여 덧셈/뺄셈 결과 출력
 - `math.js`에서 `calculator.js`를 `import`하여 덧셈/뺄셈 결과 출력

- 출력 예시

`add 결과: 7`

`sub 결과: 1`

[`calculator.js`]

// 두 수를 더하고 빼는 함수들을 모아둔 모듈

// 코드 작성 1) `add` 함수 작성 후 `export`하기

// `export function` 함수이름(매개변수) { return 값; } (`greet.js` 참고)

// 함수 이름: `add`, 매개변수: `a, b`, 반환값: `a + b`

// 코드 작성 2) `sub` 함수 작성 후 `export`하기

// 함수 이름: `sub`, 매개변수: `a, b`, 반환값: `a - b`

[`math.js`]

// `calculator.js` 모듈을 불러와서 계산 결과를 출력

// 코드 작성 3) `calculator.js`에서 `add`, `sub` 함수 불러오기

// `import { 함수이름1, 함수이름2 } from './파일명.js'` (`main.js` 참고)

// 코드 작성 4) `add(3, 4)`를 호출해서 결과를 콘솔에 출력하기

// 형태: `add 결과: 7`

// 코드 작성 5) `sub(4, 3)`을 호출해서 결과를 콘솔에 출력하기

// 형태: `sub 결과: 1`

연습문제

2. 에코 모듈 작성하기 (echo.js / server-echo.js)

URL 쿼리 문자열에 담긴 **text** 값을 그대로 돌려주는 기능을 별도 모듈로 분리하고 서버 파일에서 불러와 사용합니다.

- 설명: 요청을 처리하는 로직과 서버를 실행하는 로직을 분리해두면 나중에 처리 로직이 복잡해져도 서버 파일은 깔끔하게 유지할 수 있습니다.
- 요구사항
 - **echo.js** 모듈 작성
 - **handleEcho(req, res, query)** 함수 정의
 - **/echo?text=hello** 요청 시 **text** 값을 응답
 - **server-echo.js**에서 해당 모듈을 **import** 후 **/echo** 경로 요청이 오면 **handleEcho**를 호출해서 처리
- 출력 예시
요청: **/echo?text=안녕하세요**
응답: 안녕하세요

[echo.js]

// **/echo?text=...** 요청을 처리하는 함수 모듈

// 코드 작성 1) **handleEcho** 함수 작성 후 **export**하기

// **export function handleEcho(req, res, query) { ... }** (routes.js의 **handleRoute** 패턴 참고)

// 함수 안에서 할 일:

// - **query.get('text')**로 **text** 값 꺼내기 (query-echo.js에서 **searchParams** 다뤘던 방식)

// - **res.writeHead(200, { 'Content-Type': ... }, res.end(text 값))**으로 응답

[server-echo.js]

// **echo.js** 모듈을 불러와서 **/echo** 요청을 처리하는 서버

import http from 'node:http';

// 코드 작성 2) **echo.js**에서 **handleEcho** 함수 불러오기

// **import { 함수이름 } from './파일명.js'**

const server = http.createServer((req, res) => {

const baseUrl = `http://\${req.headers.host}`;

const url = new URL(req.url, baseUrl);

const pathname = url.pathname;

const query = url.searchParams;

// 코드 작성 3) **pathname**이 **/echo**인 경우 **handleEcho(req, res, query)** 호출하기

// 아니면 **404** 응답하기

});

server.listen(3000, () => {

console.log('http://localhost:3000 에서 실행 중');

});

연습문제

3. 사용자 인사 모듈 작성하기 (`user-router.js` / `app.js`)

`/user?name=이름` 형식의 요청을 처리하는 기능을 모듈로 분리하고 `name` 값이 있는지 없는지에 따라 다른 메시지를 응답합니다.

- 요구사항

- `user-router.js` 모듈 작성
 - `handleUser(req, res, query)` 함수 정의
 - `name` 값이 있으면 "안녕하세요, `OOO`님!" 출력
 - `name`이 없으면 "이름이 없습니다" 출력
- `app.js`에서 모듈을 불러와 `/user` 경로 요청 처리

- 출력 예시

요청: `/user?name=nodejs`

응답: 안녕하세요, `nodejs`님!

요청: `/user`

응답: 이름이 없습니다

[`user-router.js`]

```
// 코드 작성 1) handleEcho 함수 작성 후 export하기
// export function handleEcho(req, res, query) { ... } (routes.js의 handleRoute 패턴 참고)
// 함수 안에서 할 일:
// - query.get('text')로 text 값 꺼내기 (query-echo.js에서 searchParams 다뤘던 방식)
// - res.writeHead(200, { 'Content-Type': ... }), res.end(text 값)으로 응답
```

[`app.js`]

```
import http from 'node:http';
// 코드 작성 2) echo.js에서 handleEcho 함수 불러오기
// import { 함수이름 } from './파일명.js'

const server = http.createServer((req, res) => {
  const baseURL = `http://${req.headers.host}`;
  const url = new URL(req.url, baseURL);
  const pathname = url.pathname;
  const query = url.searchParams;

  // 코드 작성 3) pathname이 '/echo'인 경우 handleEcho(req, res, query) 호출하기
  // 아니라면 404 응답하기
});

server.listen(3000, () => {
  console.log('http://localhost:3000 에서 실행 중');
});
```

요청(Request) 객체의 이벤트 처리

□ req 객체란?

○ 개념 정의

- **req**는 클라이언트가 서버로 보내는 요청 정보를 담고 있는 객체
- Node.js의 기본 **http** 모듈에서 제공하며, **Express** 내부에서도 확장되어 사용
- URL, 헤더, 요청 방식, 본문(**body**) 등의 정보를 포함
- 특히 **POST**, **PUT** 등의 요청에서는 본문 데이터 수신을 위한 스트림 이벤트가 매우 중요

□ req 객체의 주요 이벤트

○ data

- 정의: 본문 데이터가 조각(**chunk**) 단위로 도착할 때마다 발생
- 용도
 - **POST**, **PUT** 요청의 **body** 데이터를 수신할 때 사용
 - 조각들을 문자열이나 버퍼로 모아서 최종 **body** 완성

```
let body = "";
req.on('data', chunk => {
  body += chunk;
});
```

○ end

- 정의: 모든 데이터 수신에 완료되었을 때 발생
- 용도
 - 조각으로 수신한 데이터를 파싱하거나 처리 로직 실행
 - **JSON** 파싱, **DB** 저장, 응답 반환 등

```
req.on('end', () => {
  const json = JSON.parse(body);
  // 데이터 처리 시작
});
```

○ error

- 정의: 요청 수신 중 오류가 발생했을 때 호출
- 용도
 - 클라이언트가 비정상적으로 연결을 끊거나 데이터 형식 오류 발생 시 처리
 - 오류 메시지 응답, 서버 로그 기록 등

```
req.on('error', err => {
  console.error('요청 중 오류 발생:', err);
  res.writeHead(400);
  res.end('Invalid request');
});
```


이벤트	정의	용도
<code>data</code>	본문 데이터 조각 수신 시 발생	<code>body</code> 조립
<code>end</code>	모든 <code>chunk</code> 수신 후 호출	파싱, 응답 처리
<code>error</code>	수신 중 오류 발생	에러 응답 및 로깅

[실습]

요청 이벤트 처리 확인

1. app-req-event.js

```
import http from 'node:http';
```

```
const server = http.createServer((req, res) => {
  let body = '';
```

```
  // 요청 데이터 수신 (chunk 단위)
```

```
  req.on('data', chunk => {
    console.log('[data] 데이터 수신 중:', chunk.toString());
    body += chunk;
  });
```

```
  // 모든 데이터 수신 완료
```

```
  req.on('end', () => {
    console.log('[end] 모든 데이터 수신 완료');
    console.log('전체 요청 본문:', body);
```

```
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    res.end(`<h2>데이터 수신 완료: ${body}</h2>`);
  });
```

```
  // 오류 발생 시
```

```
  req.on('error', err => {
    console.error('[error] 요청 중 오류 발생:', err);
    res.writeHead(400, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('요청 중 오류 발생');
  });
});
```

```
server.listen(3000, () => {
  console.log('http://localhost:3000 테스트 서버 실행 중');
});
```

2. 동작 확인

node app-req-event.js

POST 요청 본문 파싱과 JSON

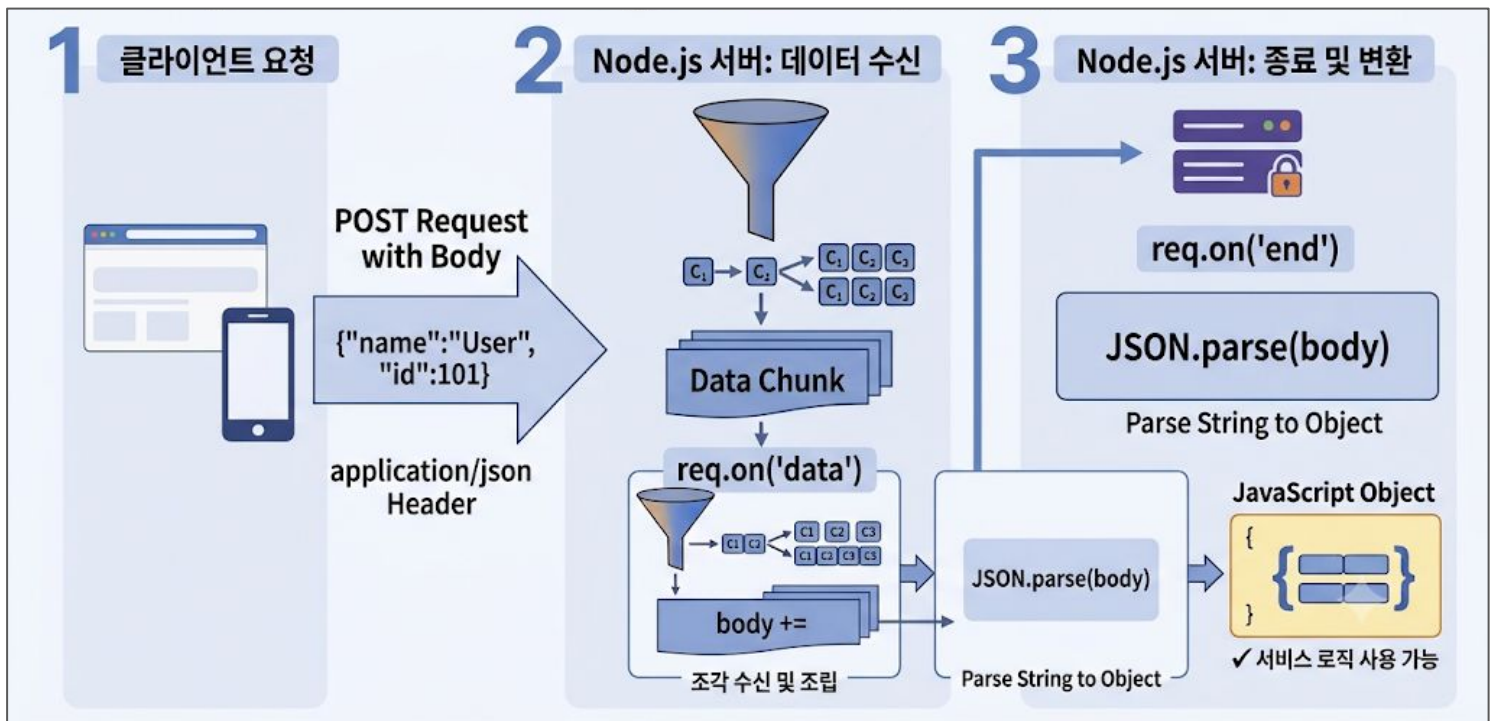
□ POST 요청과 JSON 데이터 처리

○ 개념 정의

- **GET 요청**은 데이터를 URL 주소 뒤에 실어 보내지만(Query String)
POST 요청은 데이터(회원가입, 게시글 등)를 요청 본문(Body)에 숨겨서 보냄
- 웹 개발(REST API)에서는 이 본문 데이터를 주로 **JSON(JavaScript Object Notation)** 문자열 형식으로 주고 받음
- 서버는 클라이언트가 보낸 JSON 문자열 조각들을 수신·조립한 후 자바스크립트 객체로 변환(Parsing)해야 사용할 수 있음

○ 데이터 수신 및 파싱 메커니즘 (작동 원리)

- 클라이언트 요청: **Content-Type: application/json** 헤더와 함께 JSON 데이터를 Body에 실어 전송
- 데이터 스트림 수신 (**data** 이벤트): Node.js 서버는 대용량 데이터를 한 번에 받지 않고 조각(Chunk) 단위로 나누어 받으므로, 이를 하나의 문자열로 결합하는 과정이 필요함
- 수신 완료 및 객체화 (**end** 이벤트): 모든 데이터 조각이 모이면 문자열을 자바스크립트가 읽을 수 있는



[실습]

POST JSON 데이터 수신 서버

1. app-json.js

```
import http from 'node:http';

const server = http.createServer((req, res) => {
  // 1. POST /user 요청 경로만 처리
  if (req.method === 'POST' && req.url === '/user') {
    let body = '';

    // 2. 스트림으로 들어오는 데이터 조각(chunk)들을 하나로 모음
    req.on('data', (chunk) => {
      body += chunk;
    });

    // 3. 데이터 수신에 완전히 완료(end)되었을 때 처리
    req.on('end', () => {
      try {
        // 4. 수신한 JSON 문자열을 객체로 파싱
        const userData = JSON.parse(body);
        console.log('서버에 수신된 회원 정보:', userData);

        // 5. 응답할 JSON 결과 객체 생성
        const result = {
          success: true,
          message: `${userData.name}님, 환영합니다!`
        };

        // 6. 응답 헤더를 application/json으로 설정 후 전송
        res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
        res.end(JSON.stringify(result));
      } catch (err) {
        res.writeHead(400, { 'Content-Type': 'application/json; charset=utf-8' });
        res.end(JSON.stringify({ success: false, message: '올바르지 않은 JSON 형식입니다.' }));
      }
    });
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('Not Found');
  }
});

server.listen(3000, () => {
  console.log('http://localhost:3000 에서 JSON 테스트 서버 실행 중');
});
```

2. 동작 확인

node app-json.js

METHOD

POST

SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]

http://localhost:3000/user

length: 26 byte(s)

Send

QUERY PARAMETERS

HEADERS

Form

☒ Content :

application/json

x

+ Add header

Add authorization

BODY

Text

1 {"name": "abc", "age": 20}

Text JSON XML HTML | Format body | ☒ Enable body evaluation

length: 26 bytes

Response

Cache Detected - Elapsed Time: 5ms

200 OK

HEADERS

pretty

Content-Ty... application/json; charset=utf-8

Date: Thu, 09 Jul 2026 23:44:30 GMT

Connection: keep-alive

Keep-Alive: timeout=5

Transfer-E... chunked

COMPLETE REQUEST HEADERS

BODY

pretty

{

success: true,

message: "abc님, 환영합니다!"

}

lines nums

copy

length: 53 bytes

응답(Response) 객체의 이벤트 처리

□ res 객체란?

○ 개념 정의

- **res**는 클라이언트에게 데이터를 보내는 역할을 담당하는 응답 객체
- **Node.js**의 기본 **http** 모듈 또는 **Express** 내부에서도 사용됨
- **res.write()**, **res.end()** 등을 통해 데이터를 전송함

□ res 객체의 주요 이벤트

○ finish

- 정의: 클라이언트에게 응답이 모두 전송된 후 발생하는 이벤트
- 용도: 로그 기록, 처리 시간 측정, DB 저장 등

```
res.on('finish', () => {  
  console.log('응답 완료');  
});
```

○ close

- 정의: 클라이언트와의 연결이 끊겼을 때 발생 (정상 / 오류 모두 동작)
- 주의: **res.end()** 전에 끊길 수도 있음
- 용도: 응답이 끝나지 못한 상황 감지 (사용자가 요청 중단, 브라우저를 강제로 종료하는 경우 등)

```
res.on('close', () => {  
  console.log('클라이언트와의 연결이 끊어졌습니다');  
});
```

○ error

- 정의: 응답 처리 중 내부 오류 발생 시 호출됨
- 용도: 에러 로깅, fallback 처리, 예외 응답 반환

```
res.on('error', (err) => {  
  console.error('응답 중 오류 발생:', err);  
});
```

이벤트명	발생 시점	주요 용도	주의사항
finish	응답 완료 후	응답 시간 측정, 로그	응답이 정상 완료된 경우만 발생
close	연결이 끊겼을 때	사용자 강제 중단 감지	res.end() 전에 발생 가능
error	오류 발생 시	예외 처리	Express에선 자동 처리되는 경우 많음

[실습]

finish와 close 비교

1. app-res-event.js

```
import http from 'node:http';

const server = http.createServer((req, res) => {
  res.on('finish', () => {
    console.log('[finish] 응답이 성공적으로 완료되었습니다');
  });

  res.on('close', () => {
    console.log('[close] 연결이 닫혔습니다 (중간 중단일 수 있음)');
  });

  res.on('error', (err) => {
    console.error('응답 중 오류 발생:', err);
  });

  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.write('<h1>응답을 보내는 중...</h1>');
  setTimeout(() => {
    res.end('<h2>응답 종료</h2>');
  }, 3000); // 일부러 응답 지연
});

server.listen(3000, () => {
  console.log('http://localhost:3000 테스트 중');
});
```

2. 동작 확인

node app-res-event.js



```
○ (base) ggoreb@GGoRebui-MacBookAir 03 % node app_res_event.js
http://localhost:3000 테스트 중
[finish] 응답이 성공적으로 완료되었습니다
[close] 연결이 닫혔습니다 (중간 중단일 수 있음)
█
```

순수 Node.js vs Express

□ 순수 Node.js 서버 (if문과 반복의 늪)

○ 기능이 고작 3개(메인 페이지, 회원가입, 이미지 출력)인 서버를 순수 내장 모듈로 작성한다면

```
import http from 'node:http';
import fs from 'node:fs';
import path from 'node:path';

const server = http.createServer((req, res) => {
  // 1. 라우팅의 복잡도 (if문의 연속)
  if (req.method === 'GET' && req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    res.end('<h1>메인 페이지</h1>');
  } else if (req.method === 'POST' && req.url === '/user') {
    // 2. 매번 중복되는 보일러플레이트 (데이터 수신 노가다)
    let body = '';
    req.on('data', chunk => body += chunk);
    req.on('end', () => {
      const userData = JSON.parse(body);
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ message: '가입 성공', userData }));
    });
  } else if (req.method === 'GET' && req.url === '/logo.png') {
    // 3. 정적 파일 처리의 번거로움 (MIME 타입 분기 및 파일 읽기)
    const filePath = path.join(process.cwd(), 'logo.png');
    fs.readFile(filePath, (err, data) => {
      if (err) {
        res.writeHead(404); res.end('Not Found');
      } else {
        res.writeHead(200, { 'Content-Type': 'image/png' });
        res.end(data);
      }
    });
  } else {
    res.writeHead(404); res.end('Not Found');
  }
});
server.listen(3000);
```

□ Express로 전환한 서버

- 번거로운 3가지 문제를 미들웨어와 직관적인 API를 통해 몇 줄로 해결

```
import express from 'express';
```

```
const app = express();
```

```
// 1. 보일러플레이트 해결: JSON 바디 자동 파싱 미들웨어
```

```
app.use(express.json());
```

```
// 2. 정적 파일 해결: 'public' 폴더 안의 파일(이미지 등) 자동 서빙
```

```
app.use(express.static('public'));
```

```
// 3. 직관적인 라우팅과 응답 체이닝
```

```
app.get('/', (req, res) => {
```

```
  res.send('<h1>메인 페이지</h1>'); // 내장된 Content-Type 자동 지정
```

```
});
```

```
app.post('/user', (req, res) => {
```

```
  // req.on 없이 이미 파싱된 데이터를 바로 사용!
```

```
  res.json({ message: '가입 성공', userData: req.body });
```

```
});
```

```
app.listen(3000, () => console.log('Express 서버 실행 중'));
```